Object-Oriented File Carving Tool in C++: A Signature-Based Educational Security Application

Berke Taha Özdemir
ORCID: 0009-0008-3670-0152
244410043
Department of Computer Engineering
Kastamonu University

Abstract - This study presents a simple object-oriented and command-line-based file carving application developed with C++17. The application searches a local binary file provided by the user for JPEG, PNG, and PDF file signatures; when matching start and end patterns are found, the detected fragment is saved as a separate output file. The project is not intended for malicious use or unauthorized access. Instead, it was prepared to demonstrate file carving, security tools, raw byte analysis, and object-oriented design principles in an educational context. Classes, encapsulation, abstraction, inheritance, and polymorphism are explicitly used in the design. In the tests, sample files containing a fake PDF fragment and an embedded image with a PNG signature were successfully detected.

Keywords - file carving, C++17, OOP, digital forensics, security tools, file signature, binary analysis

I. INTRODUCTION

In digital forensics and incident response processes, examining raw data plays an important role when file system information is missing, damaged, or incomplete. The file carving approach relies on characteristic signatures found inside file content rather than file names or directory records. This study converts the core idea of this approach into a student-level C++ project. Current cybersecurity guidance also emphasizes that incident response should be connected with organizational risk management and recovery activities; NIST CSF 2.0 and NIST SP 800-61 Rev. 3 are therefore used as the main contemporary security-process references [1], [2].

The aim of the project is not to develop professional disk recovery software. Instead, a demonstration tool was created that searches for JPG, PNG, and PDF signatures in a safe and controlled local test file, writes the detected fragments into separate files, and clearly shows object-oriented programming principles during this process. Recent digital-forensics process research still emphasizes traceability, documentation quality, and evidence-handling discipline for digital evidence [3]. Therefore, the application is kept within ethical and defensive boundaries.

II. CONTRIBUTION

The main contribution of this study is that it makes the file carving concept concrete through a class-based, traceable, and student-level understandable C++17 implementation. The application separates file reading, signature definition, the scanning engine, and output generation into different classes, showing how OOP principles can be used in the development of a security tool.

The study also demonstrates signature-based content analysis and its ability to make decisions independently of file extensions through small and controlled test scenarios. Limiting the tool to local and authorized test files creates a deliberate balance between the technical learning objective and the requirement for ethical use.

III. THEORETICAL BACKGROUND

A. What Is File Carving?

File carving is the extraction of file fragments by searching raw data for start and end patterns or other identifying structures that belong to specific file types. Recent work describes file carving as a content-based recovery technique used when ordinary file-system metadata is damaged, deleted, or unavailable [4], [5]. This makes it suitable for demonstrating low-level byte analysis in an educational security-tool scenario. Tools such as PhotoRec also ignore the file system and focus on the underlying data; therefore, they may be useful on damaged or reformatted media [9], [10].

The model used in this project is a simple signature-based carving approach controlled with footer patterns. For JPEG, FF D8 FF is used as the start pattern and FF D9 as the end pattern; for PNG, the standard eight-byte PNG signature and the IEND block are used; and for PDF, the %PDF- start marker and the %%EOF end marker are used. The PNG signature is defined in the W3C PNG Third Edition specification [14]. The JPEG/JFIF signature explanation is supported by the Library of Congress format description [15]. For the PDF side, the current PDF 2.0 errata and specification-maintenance notes of the PDF Association are considered [16].

B. Security Tools Concept

Security tools are software tools used to protect and analyze systems, investigate incidents, and increase security awareness. Within this scope, file carving tools can be considered supporting components of data recovery and forensic investigation processes. Recent literature continues to list PhotoRec, Foremost, Autopsy, and similar tools as examples of file carving or forensic recovery environments [4]. Autopsy is maintained as an open-source digital forensics platform, while PhotoRec remains part of the TestDisk data recovery toolset [8], [9].

Similarly, bulk_extractor can search disk images, files, or directories for structured forensic information without parsing the file system structure [11]. YARA supports rule-based recognition through textual and binary patterns, and libmagic provides another

example of magic-number-based file type recognition [12], [13]. This project should be considered a limited and educational miniature of these professional tools.

IV. PROJECT PURPOSE AND SCOPE

The developed application takes two command-line arguments: the input file path and the output folder path. The program works only on local test files provided by the user. It does not aim to process unauthorized devices, disks, network resources, or data belonging to third parties. This decision is compatible with contemporary secure-development thinking, where limited privileges, controlled inputs, and clear software boundaries are emphasized [20].

The scope is intentionally kept narrow: the tool scans only JPEG, PNG, and PDF signatures; it does not repair corrupted files, merge fragmented files, analyze file systems such as NTFS/FAT, or attempt to decrypt encrypted content. This limitation was chosen deliberately to clarify the OOP objective of the assignment and to produce a working and explainable educational demonstration.

V. CLASS DESIGN AND OOP PRINCIPLES

In the design, each class is assigned a single and clear responsibility. BinaryFileReader handles file reading, PatternUtils handles byte-pattern searching, FileSignature represents file signature abstraction, FileCarver performs the scanning engine task, and OutputWriter writes output files. Thus, the main function is kept simple and the program flow is coordinated inside the CommandLineApplication class.

TABLE I. CLASSES USED IN THE PROJECT AND THEIR RESPONSIBILITIES

BinaryFileReader: Reads the input file in binary mode and returns a byte vector.
PatternUtils: Performs pattern matching and first-match searching inside a byte sequence.
FileSignature: Abstracts the file type name, extension, and header information.
FixedFooterSignature: A common inheritance class that searches for a footer after the header.
Jpeg/Png/PdfSignature: Defines the signature information of the supported file types.
FileCarver: Scans data with registered signatures and produces a fragment list.
OutputWriter: Writes the detected fragments to the output folder as files.
CommandLineApplication: Manages command-line arguments and the general workflow.

A. Encapsulation

The internal data of the classes cannot be changed directly from the outside. For example, the signature list kept inside FileCarver is private and can only be extended through the addSignature function. This approach allows the internal state of the class to be managed in a controlled way.

B. Abstraction, Inheritance, and Polymorphism

The FileSignature abstract base class defines the common behavior of different file types. FixedFooterSignature derives from this class and centralizes the footer search logic. JpegSignature, PngSignature, and PdfSignature are concrete signature classes derived from the same base. FileCarver stores these classes inside a std::unique_ptr<FileSignature> vector; therefore, the scanning engine works through a common interface without knowing the concrete type. This structure is also consistent with the ownership and resource management approach recommended by the C++ Core Guidelines [17].

```
FileSignature
  -> FixedFooterSignature
       -> JpegSignature
       -> PngSignature
       -> PdfSignature

FileCarver
  -> vector<unique_ptr<FileSignature>>
```

Fig. 1. Summary view of class relationships.

VI. CORE ALGORITHM

The algorithm first loads the file into memory as a raw byte vector. Then, at each byte position, the registered signatures are tested in order. If the start signature matches, the footer pattern of the related file type is searched. If a valid footer is found, the data between the start and end positions is copied into a CarvedFragment object. To prevent the same fragment from being found again, the scanning position is advanced to the end of the fragment.

```
for position in data:
  for signature in signatures:
    if signature.matchesStart(data, position):
      end = signature.findEndExclusive(data, position)
      if end is valid:
        create CarvedFragment
        position = end
```

Fig. 2. Simplified pseudocode of the FileCarver scanning algorithm.

In its simple form, the runtime of this model depends on the multiplication of the data length and the number of signatures. Since the number of supported signatures is limited to three, no performance problem is expected in student-level test files. In larger systems, multi-pattern search algorithms, configurable signature databases, or modern file-fragment classification approaches may be preferred [5]. Tools such as YARA and libmagic also provide more advanced examples of pattern-based recognition [12], [13].

VII. C++17 IMPLEMENTATION DECISIONS

In the application, std::vector<unsigned char> is used to store raw file data. std::optional<std::size_t> is preferred to return an empty result when a footer cannot be found. std::unique_ptr<FileSignature> makes the ownership of signature objects explicit inside FileCarver and enables polymorphic use. The C++17 filesystem library is used to create the output folder and manage output file paths in a platform-independent way [18], [19].

To keep the code safe, the program reads only the specified file path and writes only to the specified output folder. When the input file cannot be opened, an exception is caught and a clear error message is shown to the user. This approach provides understandable error handling for command-line tools and prevents adding unnecessary privileges, network access, or offensive capabilities within the scope of the assignment [20].

VIII. TEST SCENARIOS AND FINDINGS

In the tests, a test.bin file containing only text was first used. Since this file did not contain any supported file signature, the program produced a result of 0 fragments. Then, a test_pdf.bin file containing fake PDF start and end signatures was prepared, and the program successfully extracted the PDF fragment from this file. Finally, a file named test_jpg.bin, which actually contained an embedded image with a PNG signature, was tested. Since the program checks the signature rather than the file name, it correctly produced a PNG fragment.

TABLE II. TEST SCENARIOS AND EXPECTED/ACTUAL RESULTS

1. test.bin containing only text | Expected: 0 fragments | Actual: 0 fragments found.
2. test_pdf.bin with fake PDF signatures | Expected: 1 PDF fragment | Actual: PDF start at 18, output .pdf.
3. test_jpg.bin containing an embedded image | Expected: 1 image fragment | Actual: PNG start at 18, output .png.
4. File with a missing footer | Expected: Should not produce output | Actual: No fragment is created if the footer is missing.
5. Multiple embedded files | Expected: Each should be written separately | Actual: The algorithm supports multiple fragments.

In the sample output, 9407 bytes were read from the test_jpg.bin file, 1 fragment was found, the fragment was detected as PNG, and it was written as output\fragment_001_PNG_18.png. This result shows that the tool design makes decisions according to content signatures rather than file extensions.

IX. LIMITATIONS AND ETHICAL SCOPE

This study was not designed to replace professional forensic software. The program does not parse disk images, recover deleted files at the file-system level, merge fragmented files, or repair corrupted files. It also does not attempt to decode encrypted or compressed complex structures. These limitations keep the project safe, understandable, and focused on OOP.

From an ethical use perspective, the application should be run only on local files owned by the user or created for testing purposes. This approach is consistent with recent digital-forensics process research that emphasizes evidence traceability, documentation quality, and procedural consistency [3]. In the future, a graphical interface, more file types, a configurable signature list, and detailed logging may be added; however, keeping the core engine command-line-based facilitates automation, which is common in security tools.

X. CONCLUSION

In this project, a simple and educational file carving tool based on object-oriented programming principles was developed with C++17. The program reads a local binary file received from the command line, searches for JPEG, PNG, and PDF file signatures, and writes the detected fragments into separate output files. Classes, encapsulation, abstraction, inheritance, and polymorphism are clearly applied in the design.

As a result, the application forms a university assignment demonstration suitable for showing both the file carving concept and C++ OOP design. The scope of the code was deliberately kept simple, and the security topic was handled within an ethical and defensive framework. The test outputs show that the signature-based carving logic works and that the program can detect the actual file type according to byte signatures independently of the file name.

XI. FUTURE WORK

In future work, the application may be extended with configurable signature lists, support for file types beyond JPEG/PNG/PDF, and a more scalable architecture that reads large files in chunks instead of loading all data into memory. In addition, hash generation, timestamped logging, and simple reporting outputs may be added for extracted fragments to create an educational scenario that is closer to forensic investigation processes [3], [7].

In the next stage, false positives may be reduced with YARA-like pattern definitions, libmagic-based verification mechanisms, or more advanced classification techniques for file fragments [5], [12], [13]. Such additions could preserve the current core carving engine while transforming the tool into a more sustainable, extensible, and laboratory-ready security education platform in the future.

REFERENCES

[1] NIST, “The NIST Cybersecurity Framework (CSF) 2.0,” NIST Cybersecurity White Paper 29, Feb. 26, 2024. [Online]. Available: https://csrc.nist.gov/pubs/cswp/29/the-nist-cybersecurity-framework-csf-20/final. Accessed: May 25, 2026.

[2] A. Nelson, S. Rekhi, M. Souppaya, and K. Scarfone, “Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile,” NIST SP 800-61 Rev. 3, Apr. 2025. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-61r3. Accessed: May 25, 2026.

[3] Z. Morić, V. Dakić, and I. Ogrizek Biškupić, “An Empirical Assessment of Digital Forensic Process Reliability Using Integrated ISO/IEC 27037 and 27041 Standards,” Journal of Cybersecurity and Privacy, vol. 6, no. 2, article 57, Mar. 30, 2026. [Online]. Available: https://doi.org/10.3390/jcp6020057. Accessed: May 25, 2026.

[4] P. C. Sethi, “File Carving: Analyzing Data Retrieval in Digital Forensics,” International Journal of Computer and Information Technology, vol. 13, no. 3, Sept. 2024. [Online]. Available: https://ijcit.com/index.php/ijcit/article/view/420/109. Accessed: May 25, 2026.

[5] A. Guzhov and C. T. Wirth, “Transformer-Based File Fragment Type Classification for File Carving in Digital Forensics,” in Proceedings of the 24th European Conference on Cyber Warfare and Security (ECCWS 2025), 2025. [Online]. Available: https://www.dfki.de/fileadmin/user_upload/import/16030_Guzhov_ECCWS_2025.pdf. Accessed: May 25, 2026.

[6] K. Hughes and M. Black, “An Alternative Approach to Data Carving PDF Files,” Journal of Cybersecurity Education, Research and Practice, vol. 2024, no. 1, article 21, 2024. [Online]. Available: https://digitalcommons.kennesaw.edu/jcerp/vol2024/iss1/21/. Accessed: May 25, 2026.

[7] M. Stanković and T. M. Khan, “Digital Forensics Tool Evaluation on Deleted Files,” in Digital Forensics and Cyber Crime, Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, Springer, 2023, pp. 61-83. [Online]. Available: https://doi.org/10.1007/978-3-031-36574-4_4. Accessed: May 25, 2026.

[8] The Sleuth Kit, “Autopsy User Documentation: Autopsy User’s Guide 4.22.0,” 2025. [Online]. Available: https://sleuthkit.org/autopsy/docs/user-docs/4.22.0/. Accessed: May 25, 2026.

[9] CGSecurity, “TestDisk Download: TestDisk & PhotoRec 7.2,” Feb. 22, 2024. [Online]. Available: https://www.cgsecurity.org/wiki/TestDisk_Download. Accessed: May 25, 2026.

[10] CGSecurity, “PhotoRec, Digital Picture and File Recovery,” May 21, 2023. [Online]. Available: https://www.cgsecurity.org/wiki/PhotoRec. Accessed: May 25, 2026.

[11] S. Garfinkel, “bulk_extractor,” GitHub repository, release information updated in 2024. [Online]. Available: https://github.com/simsong/bulk_extractor. Accessed: May 25, 2026.

[12] VirusTotal, “YARA Releases,” GitHub, YARA v4.5.5, Oct. 30, 2025. [Online]. Available: https://github.com/VirusTotal/yara/releases. Accessed: May 25, 2026.

[13] M. Kerrisk, “libmagic(3) - Linux manual page,” man7.org, page generated from an upstream commit dated May 17, 2026. [Online]. Available: https://man7.org/linux/man-pages/man3/libmagic.3.html. Accessed: May 25, 2026.

[14] W3C, “Portable Network Graphics (PNG) Specification (Third Edition),” W3C Recommendation, Jun. 24, 2025. [Online]. Available: https://www.w3.org/TR/png-3/. Accessed: May 25, 2026.

[15] Library of Congress, “JPEG File Interchange Format Family,” Digital Formats, last updated May 8, 2024. [Online]. Available: https://www.loc.gov/preservation/digital/formats/fdd/fdd000619.shtml. Accessed: May 25, 2026.

[16] PDF Association, “PDF 2.0 Errata Collection 2 is now available,” Jul. 25, 2024. [Online]. Available: https://pdfa.org/pdf-2-0-errata-collection-2-is-now-available/. Accessed: May 25, 2026.

[17] B. Stroustrup and H. Sutter, “C++ Core Guidelines,” Jul. 8, 2025. [Online]. Available: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines. Accessed: May 25, 2026.

[18] Microsoft, “<filesystem> functions,” Microsoft Learn, Apr. 23, 2025. [Online]. Available: https://learn.microsoft.com/en-us/cpp/standard-library/filesystem-functions. Accessed: May 25, 2026.

[19] Microsoft, “How to: Create and use unique_ptr instances,” Microsoft Learn, Apr. 13, 2026. [Online]. Available: https://learn.microsoft.com/en-us/cpp/cpp/how-to-create-and-use-unique-ptr-instances. Accessed: May 25, 2026.

[20] H. Booth, M. Souppaya, A. Vassilev, M. Ogata, M. Stanley, and K. Scarfone, “Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile,” NIST SP 800-218A, Jul. 26, 2024. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-218A. Accessed: May 25, 2026.